МИНИСТЕРСТВО НАУКИ И ОБРАЗОВАНИЯ РФ

РЫБИНСКИЙ ГОСУДАРСТВЕННЫЙ АВИАЦИОННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМ. П.А. СОЛОВЬЕВА

В.М. Комаров

Программирование на ассемблере в упражнениях и задачах

Учебное пособие

РЫБИНСК
2016

УДК 681.322 (07)

Комаров В.М. Программирование на ассемблере в упражнениях и задачах: Учебное пособие/РГАТУ. – Рыбинск, 2016. – 52 с.

В учебном пособии рассматриваются типовые задачи программирования на ассемблере и их решения с подробными пояснениями.

Пособие подготовлено на кафедре Вычислительные системы Рыбинской государственной авиационной технологической академии имени П.А. Соловьева и рекомендовано УМО РГАТУ в качестве учебного пособия для студентов направления 09.03.01 «Информатика и вычислительная техника», в том числе для студентов заочной формы обучения.

РЕЦЕНЗЕНТЫ:

Кафедра вычислительных систем Санкт-Петербургской государственной академии аэрокосмического приборостроения.

Зам. директора по новой технике ОАО «Научно-производственная фирма «Старт» Лебедев С.А.

ВВЕДЕНИЕ

В результате изучения дисциплины «Программирование на ассемблере» студенты должны понять внутреннюю организацию вычислительной системы и научиться писать программы на ассемблере, что потребуется им в освоении дальнейших дисциплин, а именно: «Организация ЭВМ и систем», «Системное программное обеспечение», «Микропроцессорные системы».

Освоение программирования на любом языке сводится к анализу программ, написанных другими, и попыткам написания своих собственных программ.

В данном методическом пособии рассмотрено решение многих типовых задач программирования на ассемблере с подробными пояснениями, которые даются в виде комментариев. Все задачи сгруппированы по темам.

Теоретические сведения приводятся в минимальном объеме, так как они изложены в учебном пособии В.М. Комарова «Микропроцессорные системы. Программирование на ассемблере», которое выдается каждому студенту.

1. ОБЩАЯ ОРГАНИЗАЦИЯ ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ. КОМАНДЫ ПЕРЕСЫЛКИ ДАННЫХ

1.1. Описание данных

Прежде чем писать команды языка для решения некоторой задачи, необходимо определить данные, которые будут в ней обрабатываться. Данные могут быть заданы двумя путями:

1) описанием с помощью директив распределения памяти;

2) физическим адресом, по которому они расположены.

В первом случае ассемблер размещает данные по своему усмотрению, а во втором случае они размещены по указанному адресу. В связи с этим рассмотренные способы задания данных являются взаимоисключающими. Поэтому при решении задач необходимо руководствоваться следующим:

1) если в условии задачи не задан физический адрес некоторых данных, то они должны быть описаны с помощью директив распределения памяти;

2) если в условии задачи задан физический адрес некоторых данных, то их описание с помощью директив недопустимо.

Задача 1.1. Описать переменные типа байт, слово, двойное слово с инициализацией соответственно на следующие значения: без инициализации, 5324h, 12345678h. Представить их размещение в памяти системы.

Решение 1.1.

```
Var 1    DB    ?           ; Байт без инициализации
Var 2    DW    5324h       ; Слово с инициализацией
Var 3    DD    12345678h   ; Двойное слово с инициализацией
```

Размещение данных в памяти приведено на рис. 1.1.

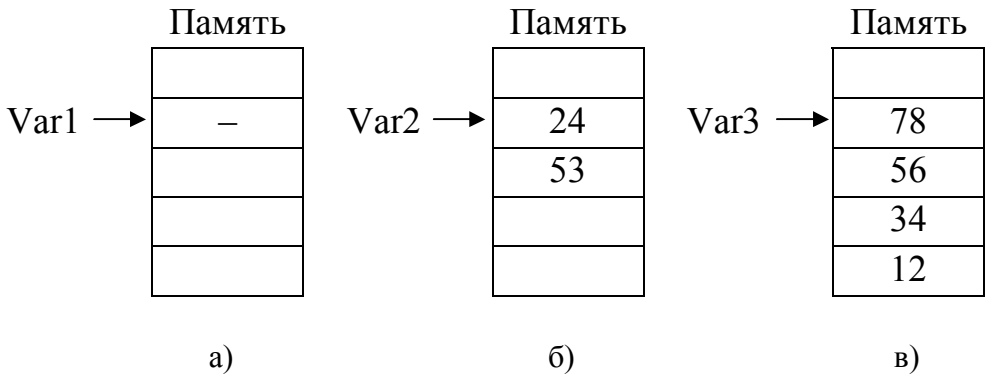


Рис. 1.1. Размещение данных в памяти: а) байт; б) слово; в) двойное слово

Задача 1.2. Описать одномерные массивы байт, слов и двойных слов. Представить их размещение в памяти системы.

Решение 1.2.

Arr 1	DB	100 DUP (0)	– одномерный массив байт из 100 элементов с инициализацией на 0;
Arr 2	DW	50 DUP (12h)	– одномерный массив слов из 50 элементов с инициализацией на значение 12h;
Arr 3	DD	25 DUP (?)	– одномерный массив двойных слов из 25 элементов без инициализации.

Размещение данных в памяти приведено на рис. 1.2. Справа указаны индексы элементов в массиве.

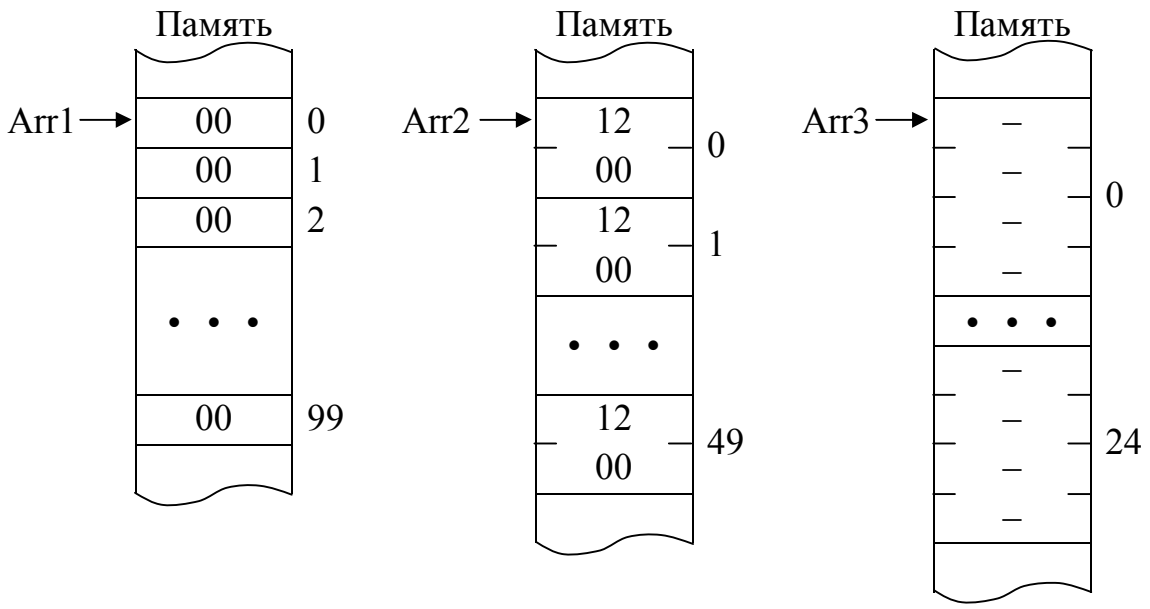


Рис. 1.2. Размещение массивов в памяти:
а) массив байтов; б) массив слов в) массив двойных слов

Индекс – это порядковый номер элемента в массиве, который всегда начинается с нуля.

Задача 1.3. Задать данные по физическим адресам ABCDEh, 525h.

Решение 1.3.

Для обращения к данным по физическому адресу этот адрес необходимо расщепить на логические компоненты сегмент: смещение, которые далее должны быть загружены в соответствующие регистры. Простейшее расщепление осуществляется по границе между цифрами физического адреса с последую-

щим дополнением каждой логической компоненты нулями до 4-х шестнадцатиричных цифр.

ABC | DE h ⇒ ABC0h: 00DEh;

00 | 525 h ⇒ 0000h: 0525h.

1.2. Команды пересылки данных

1.2.1. Команды пересылки общего назначения

К этой подгруппе относятся команды MOV, XCHG и XLAT.

Задача 1.4. Переслать значение однобайтной переменной из памяти в регистр, а содержимое двухбайтного регистра в память.

Решение 1.4.

```
VarB    DB    0              ; Описание однобайтной переменной
VarW    DW    ?              ; Описание двухбайтной переменной
        MOV   AL, VarB       ; Чтение из памяти в регистр
        MOV   VarW, AX       ; Запись из регистра в память
```

Задача 1.5. Записать число A034h в память с использованием прямой и косвенной адресации.

Решение 1.5.

```
Var     DW    ?              ; Описание двухбайтной переменной
        MOV   Var, 0A034h    ; Запись с прямой адресацией
        MOV   BX, offset Var ; Загрузка адреса
                             ; для косвенной адресации
        MOV   [BX], 0A034h   ; Запись с косвенной адресацией
```

Задача 1.6. Сохранить регистр BX в памяти с использованием прямой и косвенной адресации.

Решение 1.6.

```
Var     DW    ?              ; Описание двухбайтной переменной
        MOV   Var, BX        ; Сохранение с прямой адресацией
        MOV   DI, offset Var ; Загрузка адреса для
                             ; косвенной адресации
        MOV   [DI], BX       ; Сохранение с косвенной адресацией
```

Задача 1.7. Скопировать слово в памяти с использованием прямой и косвенной адресации.

Решение 1.7.

```
Source  DW    25h                 ; Описание источника данных
Dest    DW    ?                   ; Описание приемника данных
        MOV   AX, Source          ; Копирование с прямой
        MOV   Dest, AX            ; адресацией
        MOV   SI, offset Source   ; Загрузка адресов
        MOV   DI, offset Dest     ; для косвенной адресации
        MOV   AX, [SI]            ; Копирование с косвенной
        MOV   [DI], AX            ; адресацией
```

Задача 1.8. Скопировать двойное слово из ячейки памяти по физическому адресу ABCDEh в ячейку по физическому адресу 2373h.

Решение 1.8.

Расщепляем физические адреса на логические компоненты

A | BCDEh ⇒ A000: BCDEh

02 | 373 h ⇒ 0200 : 0373 h

```
MOV   AX, 0A000h          ; Загрузка сегментной компоненты
MOV   DS, AX              ; ячейки-источника
MOV   SI, 0BCDEh          ; Загрузка смещения
                          ; ячейки-источника
MOV   AX, 200h            ; Загрузка сегментной компоненты
MOV   ES, AX              ; ячейки-приемника
MOV   DI, 373h            ; Загрузка смещения
                          ; ячейки-приемника
MOV   AX, [SI]            ; Чтение младшего слова
                          ; из ячейки-источника
MOV   ES: [DI], AX        ; Запись младшего слова
                          ; в ячейку-приемник
MOV   AX, [SI + 2]        ; Чтение старшего слова
                          ; из ячейки-источника
MOV   ES: [DI + 2], AX    ; Запись старшего слова
                          ; в ячейку-приемник
```

В вышеприведенных задачах выполнялась обработка простых данных, то есть данных имеющих единственное значение. На практике очень часто данные

являются структурированными. Простейшим структурированным типом данных является одномерный массив. Все элементы одномерного массива расположены в смежных ячейках памяти и обрабатываются одинаково. В этом случае для упрощения программ применяется циклическая обработка данных, в которой одни и те же команды программы используются для обработки всех элементов массива. Логическая схема алгоритма циклической обработки одномерного массива приведена на рис. 1.3.

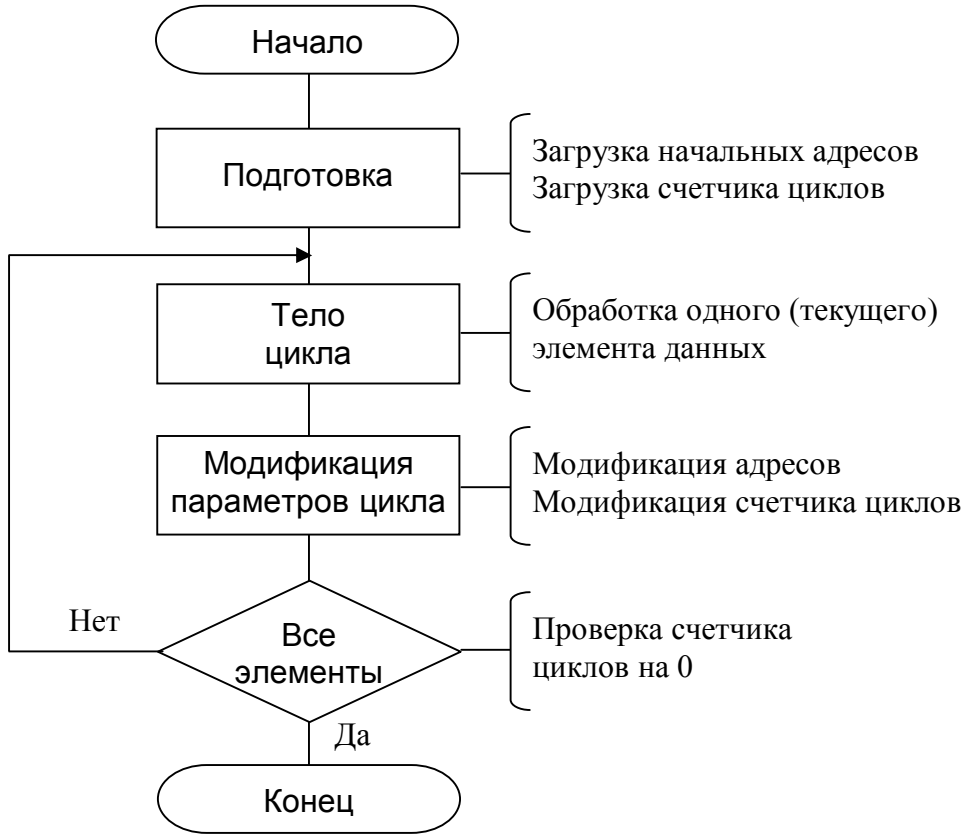


Рис. 1.3. Схема алгоритма циклической обработки одномерного массива

В операторе Подготовка загружаются начальные адреса данных и счетчик циклов на заданное количество повторений. Это обеспечивает обращение к первому элементу данных, с которого начинается обработка, и дальнейшую обработку всех элементов массива.

В операторе Тело цикла осуществляется обработка текущего элемента данных, на который указывает текущий адрес. Модификация адресов в этом операторе недопустима. Обращение ко всем фрагментам обрабатываемого элемента данных выполняется с помощью смещений к текущему адресу.

В операторе Модификация параметров цикла выполняется модификация адресов так, чтобы они указывали на следующий элемент данных, и декре-

мент счетчика циклов, учитывающий выполненную обработку очередного элемента данных.

В операторе Все элементы проверяется счетчик циклов. Если его значение неравно 0, то еще не все элементы данных обработаны, и управление передается на тело цикла, то есть обработку следующего элемента данных. Если значение счетчика циклов равно 0, то все элементы данных обработаны, и решение задачи прекращается.

Задача 1.9. Скопировать массив слов из основного сегмента данных в дополнительный сегмент.

Решение 1.9.

```
ArrS    DW      100   DUP(?)          ; Описание массива-источника
ArrD    DW      100   DUP(?)          ; Описание массива-приемника
        MOV     SI, offset ArrS       ; Загрузка начальных
        MOV     DI, offset ArrD       ; адресов массивов
        MOV     CX, length ArrS       ; Загрузка счетчика циклов
M1:     MOV     AX, [SI]              ; Копирование текущего
        MOV     ES: [DI], AX          ; элемента
        ADD     SI, type ArrS         ; Модификация адресов
        ADD     DI, type ArrD
        DEC     CX                    ; Модификация счетчика циклов
        JNZ     M1                    ; Проверка счетчика циклов и
                                      ; передача управления на метку
                                      ; M1, если он не равен 0.
```

Задача 1.10. В памяти имеются два массива слов. Обменять элементы с четными индексами массива-источника с элементами с ближайшим нечетным индексом массива-приемника.

Решение 1.10.

```
ArrS    DW      100   DUP(?)                ; Описание данных
ArrD    DW      100   DUP(?)
        MOV     SI, offset ArrS             ; Загрузка адреса
                                            ; массива-источника
                                            ; на элемент с четным
                                            ; (нулевым) индексом
        MOV     DI, offset ArrD + type ArrD ; Загрузка адреса
                                            ; массива-приемника
```

```
                                        ; на ближайший элемент
                                        ; с нечетным (равным 1)
                                        ; индексом
        MOV     CX, (length ArrS)/2     ; Загрузка счетчиков циклов
M1:     MOV     AX, [SI]                ; Чтение элемента источника
        XCHG    AX, [DI]                ; Обмен с элементом
                                        ; приемника
        MOV     [SI], AX                ; Запись элемента источника
        ADD     SI, 2* type ArrS        ; Модификация адресов
        ADD     DI, 2* type ArrD        ; через элемент
        DEC     CX                      ; Модификация счетчика
                                        ; циклов
        JNZ     M1                      ; Переход, если не все
                                        ; элементы
```

Задача 1.11. В памяти имеется массив байтов со значениями 1, 6, 13, 22, 33. Вычислить таблично $y = \sqrt{x+3}$ для всех элементов этого массива.

Решение 1.11.

Табличное преобразование данных осуществляется по таблице, в которой находятся значения решений. При этом решения должны располагаться в ячейках таблицы, индексы которых равны значениям аргумента.

```
X       DB      100   DUP (?)           ; Описание входного и
Y       DB      100   DUP (?)           ; выходного массива
        ; Описание таблицы преобразования
Tabl    DB      ?, 2, 4 DUP(?), 3, 6 DUP(?), 4, 8 DUP(?), 5, 10 DUP(?), 6
        MOV     SI, offset X            ; Загрузка адресов
        MOV     DI, offset Y            ; данных
        MOV     BX, offset Tabl         ; Загрузка адреса таблицы
M1:     MOV     AL, [SI]                ; Чтение байта из X
        XLAT    Tabl                    ; Преобразование байта
        MOV     [DI], AL                ; Запись результата в Y
        INC     SI                      ; Модификация адресов
        INC     DI
        DEC     CX                      ; Модификация счетчика
        JNZ     M1                      ; циклов и переход, если не 0
```

1.2.2. Команды загрузки адресных значений и обращения к стеку

Командами загрузки адресных значений являются команды LEA, LDS и LES. Основной командой является команда LEA, загружающая в указанный регистр адрес переменной по ее имени. Эта команда аналогична команде MOV с директивой offset.

Команды LDS и LES загружают полный логический адрес переменной из заранее подготовленного адресного буфера. Адресный буфер всегда представляет собой двойное слово в памяти системы. Для его использования он должен быть описан как обычная переменная и инициализирован на полный логический адрес переменной, которая будет адресоваться через этот буфер. Инициализация буфера может выполняться описанием либо программным путем.

Командами обращения к стеку являются команды PUSH и POP. Стек используется для временного хранения данных и адресуется регистрами SS:SP. Для записи данных в стек используются команды PUSH, а для чтения из стека команды POP. Обращение к стеку как к ОЗУ с произвольным доступом выполняется через регистр BP обычным образом.

Задача 1.12. В памяти имеется массив байтов со значениями 1, 5, 8, 13, 15. Для элементов с нечетными индексами этого массива вычислить таблично $y = x^2 + 1$. Результат разместить в текущем стеке, начиная от верхушки.

Решение 1.12.

Стек всегда существует в системе и поэтому не описывается. Для адресации к стеку необходимо скопировать содержимое регистра SP в регистр BP. Элементами стека всегда являются слова.

```
X       DB      100   DUP (?)        ; Описание входного массива данных
        ; Описание таблицы преобразования
Tabl    DB      ?, 2, 3 DUP(?), 26, 2 DUP(?), 65, 4 DUP(?), 170, ?, 226
        MOV     CX, (length X)/2     ; Загрузка счетчика циклов
        LEA     BX, X + type X       ; Загрузка адреса входного массива
                                     ; на элемент с нечетным индексом
        MOV     BP, SP               ; Привязка к верхушке стека
        MOV     AH, O                ; Расширение результата до слова
M1:     MOV     AL, [BX]             ; Чтение элемента входного массива
        PUSH    BX                   ; Освобождение BX – сохранение
                                     ; в стеке
        LEA     BX, Tabl             ; Загрузка адреса таблицы
                                     ; преобразования
        XLAT    Tabl                 ; Преобразование байта
        MOV     [BP], AX             ; Запись результата в стек
```

```
        POP     BX              ; Восстановление адреса входного
                                ; массива из стека
        ADD     BX, 2*type X    ; Модификация адреса входного
                                ; массива через элемент и адреса
        ADD     BP, 2           ; стека на следующий элемент
        DEC     CX              ; Модификация счетчика
        JNZ     M1              ; циклов и переход, если не 0.
```

Задача 1.13. Скопировать область памяти, начиная с физического адреса 9ABCDh, в массив в дополнительном сегменте данных. Для инициализации адресов памяти использовать заранее подготовленные адресные буферы. При этом один адресный буфер инициализировать описанием, а второй – программным путем.

Решение 1.13.

В данной задаче необходимо адресовать две области памяти – по физическому адресу и в дополнительном сегменте данных. Следовательно, потребуется два адресных буфера. Будем считать, что адресный буфер на массив в дополнительном сегменте памяти будет инициализирован описанием, а адресный буфер на физический адрес – программным путем. Для этого необходимо расценить заданный физический адрес на логические компоненты. Пусть расщепление адреса имеет вид 9 | ABCDh⇒9000h:ABCDh.

```
ArrD        DW      100 DUP(?)          ; Описание массива-приемника
                                        ; в памяти
AddrDBuff   DD      ArrD                ; Описание адресного буфера
                                        ; с инициализацией
                                        ; на массив-приемник
AddrFBuff   DD      ?                   ; Описание адресного буфера
                                        ; под физический адрес
                                        ; без инициализации
            MOV     AddrFBuff, 0ABCDh   ; Инициализация адресного
            MOV     AddrFBuff + 2, 9000h ; буфера на физический адрес
                                        ; программным путем
            LDS     SI, AddrFBuff       ; Загрузка адресных
            LES     DI, AddrDBuff       ; указателей
            MOV     CX, length ArrD     ; Загрузка счетчика циклов
M1:         MOV     AX, [SI]            ; Чтение элемента из источника
            MOV     ES: [DI], AX        ; Запись элемента в приемник
            ADD     SI, 2               ; Инициализация адреса
                                        ; массива-источника
```

```
        ADD     DI, type ArrD           ; Инициализация адреса
                                        ; массива-приемника
        DEC     CX                      ; Модификация счетчика
        JNZ     M1                      ; циклов и переход, если не 0
```

1.2.3. Команды ввода/вывода

Команды ввода/вывода допускают прямую и косвенную адресацию портов. Прямая адресация используется при нециклической обработке, а косвенная – при циклической обработке. При косвенной адресации модификация номеров портов осуществляется идентично модификации адресов памяти.

Задача 1.14. Вывести в порт индикатора код для отображения некоторого символа из ячейки памяти.

Решение 1.14.

```
IndPort = 125                           ; Назначение номера порту
Simv    DB      ?                       ; Описание ячейки-источника
        MOV     AL, Simv                ; Чтение символа из памяти
        OUT     IndPort, AL             ; Выбор символа в порт
```

Задача 1.15. Ввести из массива двухбайтных портов 200 слов и разместить их в памяти. Номер начального порта равен 5000h.

Решение 1.15.

```
BeginPort = 5000h                       ; Назначение номера начальному порту
Arr     DW      200 DUP(?)              ; Описание массива-приемника
        LEA     BX, Arr                 ; Загрузка начального адреса
                                        ; массива-приемника
        MOV     DX, BeginPort           ; Загрузка начального номера порта
M1:     IN      AX, DX                  ; Ввод слова из порта
        MOV     [BX], AX                ; Запись слова в массив-приемник
        ADD     BX, type Arr            ; Модификация адресов
        ADD     DX, 2
        DEC     CX                      ; Зацикливание
        JNZ     M1
```

1.3. Контрольные задачи

1.3.1. В памяти имеется массив слов. Обнулить его элементы с нечетными индексами.

1.3.2. В памяти имеется массив двойных слов. Записать строку “МИР” во все элементы этого массива.

1.3.3. Скопировать элементы массива слов из дополнительного сегмента данных в область памяти по физическому адресу BCD0Ah.

1.3.4. Скопировать массив двойных слов в памяти системы.

1.3.5. В памяти имеется массив двойных слов. Обменять его элементы с четными и нечетными индексами в пределах пар.

1.3.6. В памяти имеется массив байтов со значениями 1, 4, 9, 16, 25, 36. Для элементов с нечетными индексами этого массива вычислить $y = \sqrt{x} + 3$.

1.3.7. Начиная от верхушки скопировать 200 элементов текущего стека в область памяти по физическому адресу 321h.

1.3.8. В памяти имеется массив байтов X со значениями 1, 8, 27, 64, 125. Для всех элементов этого массива вычислить $y = \sqrt[3]{x}$, где Y – массив слов. Для инициализации адресов памяти использовать заранее подготовленные адресные буферы. При этом один буфер инициализировать описанием, а другой – программным путем.

1.3.9. Скопировать область памяти, начиная с физического адреса ABCDEh в область памяти, начиная с физического адреса 345h. Для инициализации адресов памяти использовать заранее подготовленные адресные буферы. При этом один буфер инициализировать описанием, а другой – программным путем.

1.3.10. Вывести массив слов из памяти системы в массив двухбайтных портов, начиная с порта с номером 5000h.

2. АРИФМЕТИЧЕСКИЕ КОМАНДЫ

Арифметические команды обеспечивают выполнение всех арифметических операций над данными различных типов.

2.1. АРИФМЕТИЧЕСКИЕ ОПЕРАЦИИ НАД ДВОИЧНЫМИ ЧИСЛАМИ

2.1.1. Сложение (вычитание) двоичных чисел без знака

В числах без знака все цифры числа хранят его модуль. Поэтому возникающие при переполнении разрядной сетки переносы также являются цифрами модуля, из которых должна формироваться старшая часть результата.

Имеется два граничных случая сложения чисел:

1) сложение двух чисел повышенной точности;

2) сложение множества коротких чисел (базовых элементов – байтов или слов).

Схема сложения двух чисел повышенной точности приведена на рис. 2.1.

Сложение всех элементов чисел должно выполняться циклически с учетом переноса, порожденного сложением предыдущих элементов. При этом для модификации адресов нельзя использовать команду ADD, изменяющую флаг переноса. При сложении самых старших элементов возникает последний перенос, который также необходимо подобрать. Поэтому размер результата на один элемент длиннее исходных чисел.

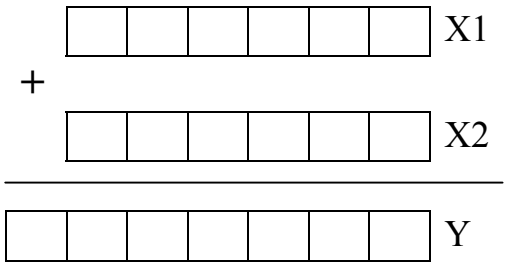


Рис. 2.1. Сложение двух чисел с повышенной точностью

Задача 2.1. Вычислить Y = X1 + X2, где X1, X2 – числа повышенной точности без знака.

Решение 2.1.

```
X1      DW      100          DUP(?)     ; Описание исходных чисел
X2      DW      100          DUP(?)
Y       DW      (100 + 1)    DUP(?)     ; Описание результата
        LEA     SI, X1                  ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
        MOV     CX, length X1           ; Загрузка счетчика циклов
        CLC                             ; Очистка флага переноса
```

```
M1:     MOV     AX, [SI]        ; Чтение элемента числа X1
        ADC     AX, [DI]        ; Сложение с элементом числа X2
        MOV     [BX], AX        ; Запись элемента результата
        INC     SI              ; Модификация адресов
        INC     SI              ; без изменения флага переноса
        INC     DI
        INC     DI
        INC     BX
        INC     BX
        DEC     CX              ; Зацикливание
        JNZ     M1
        MOV     AX, 0           ; Подбор последнего переноса
        ADC     AX, 0
        MOV     [BX], AX
```

Схема сложения множества коротких чисел приведена на рис. 2.2. При сложении каждого очередного числа с накопителем может возникать перенос, который необходимо подбирать. Из этих переносов складывается старшая часть результата. Длина результата равна длине исходных чисел плюс длина подборщика переносов. Для простоты считают, что перенос рождается при сложении каждого числа. Поэтому длина подборщика переносов определяется количеством складываемых чисел. Практически всегда при обработке двоичных чисел в качестве подборщика переноса может использоваться слово.

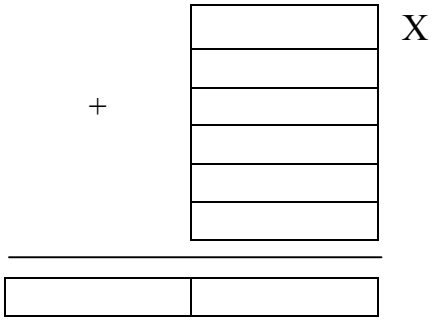


Рис. 2.2. Сложение множества коротких чисел

Задача 2.2. Вычислить сумму элементов массива слов без знака.

Решение 2.2.

```
X       DW      100   DUP(?)    ; Описание исходного массива
Y       DD      ?               ; Описание результата
        LEA     SI, X           ; Загрузка начального адреса
        MOV     CX, length X    ; Загрузка счетчика циклов
        MOV     AX, 0           ; Обнуление накопителя
        MOV     DX, AX
M1:     ADD     AX, [SI]        ; Сложение числа с накопителем
```

```
        ADC     DX, 0                   ; Подбор переноса
        ADD     SI, type X              ; Модификация адреса
        DEC     CX                      ; Зацикливание
        JNZ     M1
        MOV     word ptr Y, AX          ; Запись результата в память
        MOV     word ptr Y + 2, DX
```

2.1.2. Сложение (вычитание) двоичных чисел со знаком

В числах со знаком в старшем бите числа находится знак, а во всех остальных цифрах его модуль. Поэтому возникающие при переполнении разрядной сетки переносы не имеют физического смысла, что не позволяет использовать их для формирования старшей части результата. В связи с этим, при выполнении операций над числами со знаком переносы из знакового разряда недопустимы. Для исключения переполнений необходимо перед операцией сложения (вычитания) операнды расширить до размера результата. После этого знаковые числа обрабатываются аналогично числам без знака.

Задача 2.3. Вычислить Y = X1 + X2, где X1, X2 – числа повышенной точности со знаком.

Решение 2.3.

```
X1      DW      100         DUP(?)  ; Описание исходных чисел
X2      DW      100         DUP(?)
Y       DW      (100 + 1)   DUP(?)  ; Описание результата
        LEA     SI, X1              ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
        MOV     CX, length X-1      ; Загрузка счетчика циклов
        CLC                         ; Очистка флага переноса
M1:     MOV     AX, [SI]            ; Чтение элемента числа X1
        ADC     AX, [DI]            ; Сложение с элементом числа X2
        MOV     [BX], AX            ; Запись элемента результата
        INC     SI                  ; Модификация адресов
        INC     SI                  ; без изменения флага переноса
        INC     DI
        INC     DI
        INC     BX
        INC     BX
        DEC     CX                  ; Зацикливание
```

```
        JNZ     M1
        MOV     AX, [SI]            ; Расширение старшего
        CWD                         ; элемента числа X1
        MOV     [BX], AX            ; Сохранение расширенного
        MOV     [BX + 2], DX        ; значения в ячейке результата
        MOV     AX, [DI]            ; Расширение старшего
        CWD                         ; элемента числа X2
        ADC     [BX], AX            ; Сложение старших элементов чисел
        ADC     [BX + 2] DX         ; X1 и X2 с записью результата
                                    ; в память
```

Задача 2.4. Вычислить сумму элементов массива слов со знаком.

Решение 2.4.

```
X       DW      100    DUP(?)       ; Описание исходного массива
Y       DD      ?                   ; Описание результата
        LEA     SI, X               ; Загрузка начального адреса
        MOV     CX, length X        ; Загрузка счетчика циклов
        MOV     BX, 0               ; Обнуление накопителя
        MOV     BP, 0
M1:     MOV     AX, [SI]            ; Чтение очередного числа
        CWD                         ; Расширение числа
        ADD     BX, AX              ; Сложение числа
        ADC     BP, DX              ; с накопителем
        ADD     SI, type X          ; Модификация адреса
        DEC     CX                  ; Зацикливание
        JNZ     M1
        MOV     word ptr Y, BX      ; Запись результата
        MOV     word ptr Y + 2, BP  ; в память
```

2.1.3. Умножение двоичных чисел

Умножение двоичных чисел без знака и со знаком выполняется аналогично с помощью команд MUL и IMUL.

Однако напрямую эти команды позволяют получить результат лишь при умножении базовых элементов, то есть байтов или слов. При умножении более длинных элементов результат формируется как сумма частичных произведений.

Задача 2.5. Вычислить Y = X1*X2, где X1, X2 – массивы слов без знака.

Решение 2.5.

```
X1      DW      100   DUP (?)        ; Описание исходных массивов
X2      DW      100   DUP (?)
Y       DD      100   DUP (?)        ; Описание результата
        MOV     CX, length X1        ; Загрузка счетчиков циклов
        LEA     SI, X1               ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
M1:     MOV     AX, [SI]             ; Чтение числа X1
        MUL     word ptr [DI]        ; Умножение на число X2
        MOV     [BX], AX             ; Запись результата
        MOV     [BX + 2], DX
        ADD     SI, type X1          ; Модификация адресов
        ADD     DI, type X2
        ADD     BX, type Y
        DEC     CX                   ; Зацикливание
        JNZ     M1
```

Задача 2.6. Вычислить Y = X1∗X2, где X1, X2 – массивы двойных слов без знака.

Решение 2.6.

Элементы массивов X1 и X2 длиннее базовых элементов. Поэтому результат умножения должен формироваться последовательно как сумма частичных произведений в соответствии со схемой, приведенной на рис. 2.3.

Каждое частичное произведение будет формироваться в регистрах DX, AX. Поэтому перед вычислением следующего частичного произведения предыдущее произведение должно быть сохранено в памяти. Для организации циклических вычислений это сохранение необходимо выполнять операцией сложения, что требует обнуления накопителя результата в памяти перед выполнением операции.

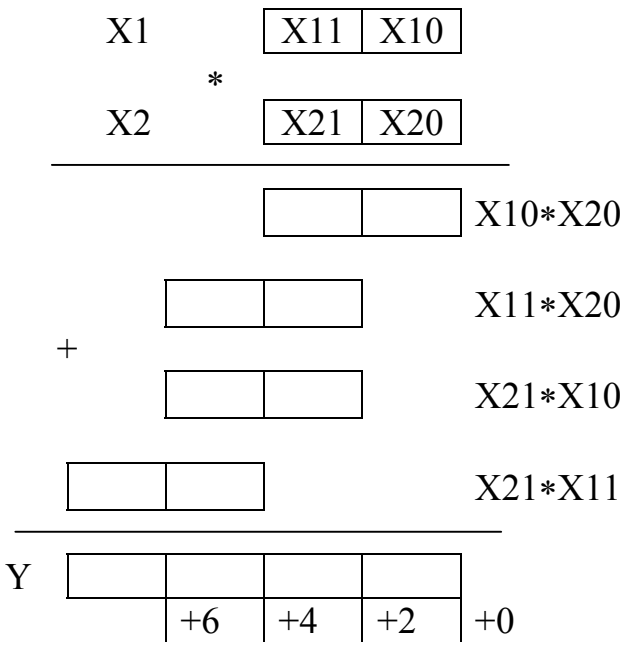


Рис. 2.3. Схема последовательного умножения двойных слов

```
X1      DD      100 DUP (?)         ; Описание входных массивов
X2      DD      100 DUP (?)
Y       DQ      100 DUP (0)         ; Описание массива результата
                                    ; с обнулением
        LEA     SI, X1              ; Загрузка
        LEA     BX, X2              ; начальных
        LEA     DI, Y               ; адресов
        MOV     CX, length X1       ; и счетчика циклов
        MOV     AX, [SI]            ; Чтение элемента X10
        MUL     word ptr [BX]       ; Умножение на X20
        ADD     [DI], AX            ; Сложение частичного произведения
        ADC     [DI + 2], DX        ; X10∗X20 с накопителем результата
        MOV     AX, [SI + 2]        ; Чтение элемента X11
        MUL     word ptr [BX]       ; Умножение на X20
        ADD     [DI + 2], AX        ; Сложение частичного произведения
        ADC     [DI + 4], DX        ; X11∗X20 с накопителем результата
        MOV     AX, [SI]            ; Чтение элемента X10
        MUL     word ptr [BX + 2]   ; Умножение на X21
        ADD     [DI + 2], AX        ; Сложение частичного произведения
        ADC     [DI + 4], DX        ; X21∗X10 с накопителем результата
        MOV     AX, [SI + 2]        ; Чтение элемента X11
```

```
        MUL     word ptr [BX + 2]   ; Умножение на X21
        ADD     [DI + 4], AX        ; Сложение частичного произведения
        ADC     [DI + 6], DX        ; X21∗X11 с накопителем результата
        ADD     SI, type X1         ; Модификация адреса
        ADD     BX, type X2
        ADD     DI, type Y
        DEC     CX                  ; и счетчика циклов
        JNZ     M1                  ; Зацикливание
```

Из анализа текста задачи следует, что все частичные произведения вычисляются и обрабатываются с помощью одинаковых команд. Это дает возможность для циклической реализации умножения чисел повышенной точности. Рассмотренный алгоритм и схема решения задачи могут быть использованы для умножения чисел любой длины.

Задача 2.7. Вычислить Y = X1∗X2, где X1, X2 – числа повышенной точности без знака.

Решение 2.7.

Поскольку числа X1 и X2 длиннее базовых элементов, то результат их умножения должен формироваться как сумма частичных произведений в соответствии со схемой, приведенной на рис. 2.4.

Из анализа этой схемы следует, что при обращении к Y при вычислении суммы частичных произведений индекс выбираемого элемента определяется суммой индексов умножаемых элементов X1 и X2. Если для адресации всех данных используется индексная адресация, и операнды записываются в виде X1[SI], X2[DI], то адрес элемента полного произведения должен записываться в виде Y[SI + DI]. Однако, такая адресация невозможна, и запись операнда необходимо свести к виду Y[BX + SI]. Для этого необходимо копировать один из индексных регистров в регистр BX и выполнять обращение к элементу полного произведения.

```
X1      DW      100 DUP(?)          ; Описание исходных
X2      DW      50  DUP(?)          ; массивов
Y       DW      150 DUP(0)
        MOV     SI, 0               ; Обнуление индекса X2
        MOV     CH, length X2       ; Загрузка счетчика внешнего цикла
M2:     MOV     DI, 0               ; Обнуление индекса X1
        MOV     CL, length X1       ; Загрузка счетчика внутреннего цикла
        MOV     BX, SI              ; Копирование индекса X2
```

```
M1:  MOV   AL, X1 [DI]          ; Чтение элемента X1
     MUL   X2[SI]               ; Умножение на элемент X2
     ADD   Y[BX + DI], AX       ; Вычисление полного
     ADC   Y[BX + DI + 2], DX   ; произведения
     ADD   DI, type X2          ; Модификация адреса
                                ; внутреннего цикла
     DEC   CL                   ; Все элементы внутреннего цикла ?
     JNZ   M1                   ; Переход, если нет
     ADD   SI, type X1          ; Модификация адреса внешнего цикла
     DEC   CH                   ; Все элементы внешнего цикла
     JNZ   M2                   ; Переход, если нет
```

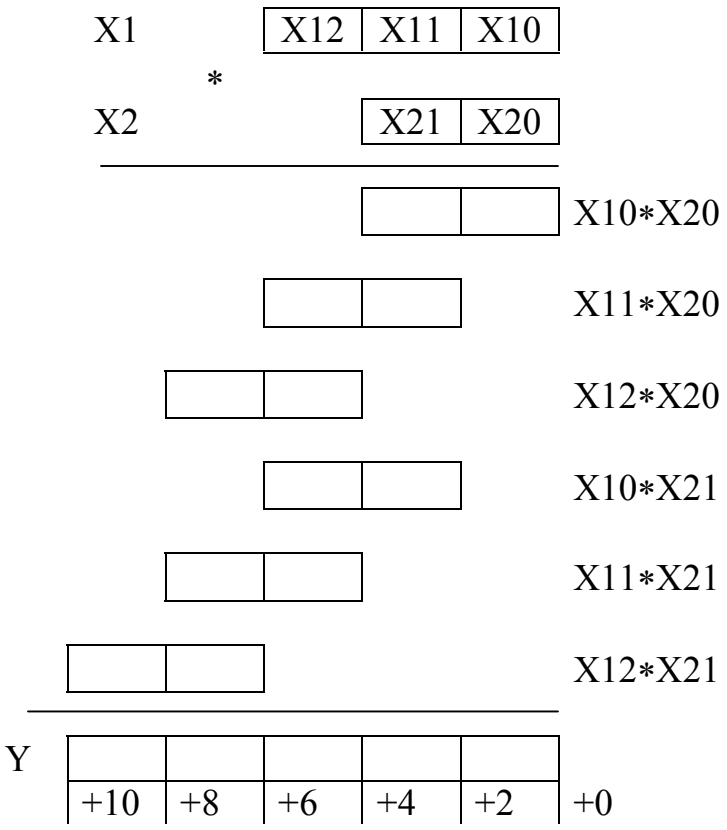


Рис. 2.4. Схема последовательного умножения чисел повышенной точности

2.1.4. Деление двоичных чисел

Деление двоичных чисел без знака и со знаком выполняется аналогично с помощью команд DIV и IDIV.

Однако напрямую эти команды дают результат лишь при делении элементов, не длиннее стандартных, и то только тогда, когда частное помещается в

регистр-приемник. Если частное не помещается в регистр-приемник, то вызывается подпрограмма обработки прерывания по ошибке деления, которая должна принять все необходимые решения. Причиной возникновения ошибки является деление двух цифр на одну. Для исключения возможности возникновения ошибки при делении чисел без знака необходимо делить одну цифру на одну. При этом деление должно выполняться последовательно цифра за цифрой с формированием частного и остатка.

Задача 2.8. Вычислить Y = X1/X2, где X1 – массив двойных слов со знаком, X2 – массив слов со знаком.

Решение 2.8.

```
X1      DD      100   DUP (?)      ; Описание исходных массивов
X2      DW      100   DUP (?)
Y       DW      100   DUP (?)      ; Описание массива частных
R       DW      100   DUP (?)      ; Описание массива остатков
        LEA     SI, X1             ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
        LEA     BP, R
        MOV     CX, length X1      ; Загрузка счетчика циклов
M1:     MOV     AX, [SI]           ; Чтение числа X1
        MOV     DX, [SI + 2]
        IDIV    word ptr [DI]      ; Деление на число X2
        MOV     [BX], AX           ; Запись частного
        MOV     DS: [BP], DX       ; Запись остатка
        ADD     SI, type X1        ; Модификация адресов
        ADD     DI, type X2
        ADD     BX, type Y
        ADD     BP, type R
        DEC     CX                 ; Зацикливание
        JNZ     M1
```

Задача 2.9. Вычислить Y = X1/X2, где X1 – число повышенной точности без знака, X2 – слово без знака.

Решение 2.9.

По правилам арифметики деление должно начинаться со старшей цифры делимого, а результат формируется со старшей цифры частного.

```
X1      DW      50 DUP(?)          ; Описание делимого
X2      DW      ?                  ; Описание делителя
Y       DW      50 DUP (?)         ; Описание частного
R       DW      ?                  ; Описание остатка
        LEA     SI, X1             ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
        LEA     BP, R
        MOV     CX, length X1      ; и счетчика циклов

; Смещение адресов делимого и частного на старшую цифру
        ADD     SI, (length X1 – 1) *type X1
        ADD     BX, (length Y – 1) *type Y
        MOV     DX, 0              ; Обнуление старшей части
                                   ; первого элемента делимого
M1:     MOV     AX, [SI]           ; Чтение элемента делимого
        MOV     word ptr [DI]      ; Деление
        MOV     [BX], AX           ; Запись цифры частного
        SUB     SI, type X1        ; Модификация адресов
        SUB     BX, type Y         ; делимого и частного
        DEC     CX                 ; Зацикливание
        JNZ     M1
        MOV     DS: [BP], DX       ; Запись остатка
```

Рассмотренный алгоритм справедлив для делимого произвольной длины и делителя в виде одного базового элемента. Решение такой задачи рассмотрено в разделе 6.

2.2. Арифметические операции над десятичными числами.

Арифметические операции над десятичными числами выполняются аналогично соответствующим операциям над двоичными числами. Однако при обработке десятичных чисел необходимо учитывать следующие особенности:

1) десятичное число всегда является байтом или последовательностью байтов;

2) результат выполнения любой операции над десятичными числами должен быть получен в регистре AL;

3) выполнение десятичных операций сложения, вычитания и умножения сводится к выполнению соответствующих команд двоичной арифметики с по-

следующей десятичной коррекцией, а выполнение десятичной операции деления сводится к предшествующей коррекции делимого с последующим выполнением команды двоичного деления.

Десятичные числа могут быть представлены в упакованном или распакованном форматах. В упакованном формате в каждом байте хранится две десятичных цифры, а в распакованном формате – одна цифра (в младшей тетраде). Десятичные числа всегда являются числами без знака.

2.2.1. Сложение десятичных чисел

Задача 2.10. Вычислить Y = X1 + X2, где X1, X2 – 20-разрядные десятичные числа в упакованном формате.

Решение 2.10.

```
X1      DB      10          DUP(?)      ; Описание исходных данных
X2      DB      10          DUP(?)
Y       DB      (10 + 1)    DUP(?)      ; Описание результата
        LEA     SI, X1                  ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
        MOV     CX, length X1           ; Загрузка счетчика циклов
        CLC                             ; Очистка флага переноса
M1:     MOV     AL, [SI]                ; Чтение байта числа X1
        ADC     AL, [DI]                ; Сложение с байтом числа X2
        DAA                             ; Коррекция в упакованном формате
        MOV     [BX], AL                ; Запись байта результата
        INC     SI
        INC     DI
        INC     BX
        DEC     CX                      ; Зацикливание
        JNZ     M1
        MOV     AL, 0                   ; Подбор переноса
        ADC     AL, 0
        DAA
        MOV     [BX], AL                ; Запись старшего байта результата
```

Задача 2.11. Найти сумму элементов массива 4-разрядных десятичных чисел в упакованном формате.

Решение 2.11.

```
X       DB      50 DUP (2 DUP(?))  ; Описание входного массива
Y       DB      4   DUP(?)         ; Описание результата
        LEA     SI, X              ; Загрузка адреса входного массива
        MOV     CX, length X       ; Загрузка счетчика цикла
        MOV     BX, 0              ; Обнуление накопителя
        MOV     DX, 0
M1:     MOV     AL, BL             ; Суммирование младшего
        ADD     AL, [SI]           ; байта числа с накопителем
        DAA
        MOV     BL, AL
        MOV     AL, BH             ; Суммирование старшего
        ADC     AL, [SI + 1]       ; байта числа с накопителем
        DAA
        MOV     BH, AL
        MOV     AL, DL
        ADC     AL, 0              ; Подбор переноса
        DAA                        ; в младший байт
        MOV     DL, AL
        MOV     AL, DH             ; Подбор переноса
        ADC     AL, 0              ; в старший байт
        DAA
        MOV     DH, AL
        INC     SI                 ; Модификация адреса
        INC     SI
        DEC     CX                 ; Зацикливание
        JNZ     M1
        MOV     word ptr Y, BX     ; Запись результата
        MOV     word ptr Y + 2, DX
```

Сложение (вычитание) десятичных чисел в распакованном формате выполняется аналогично.

Умножение и деление десятичных чисел возможно только в распакованном формате.

2.2.2. Умножение десятичных чисел

Задача 2.12. Вычислить Y = X1*X2, где X1, X2 – массивы одноразрядных десятичных чисел в распакованном формате.

Решение 2.12.

```
X1      DB      100  DUP(?)            ; Описание входных
X2      DB      100  DUP(?)            ; массивов
Y       DB      100  DUP(2 DUP(?))     ; Описание результата
        LEA     SI, X1                 ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     BX, Y
        MOV     CX, length X1          ; Загрузка счетчика циклов
M1:     MOV     AL, [SI]               ; Чтение числа X1
        MUL     byte ptr [DI]          ; Умножение на число X2
        AAM                            ; Коррекция умножения
        MOV     [BX], AX               ; Запись результата
        INC     SI                     ; Модификация адресов
        INC     DI
        ADD     BX, type Y*2
        DEC     CX                     ; Зацикливание
        JNZ     M1
```

2.2.3. Деление десятичных чисел

Задача 2.13. Вычислить Y = X1/X2, где X1 – массив двухразрядных десятичных чисел в распакованном формате, X2 – массив одноразрядных десятичных чисел в распакованном формате.

Решение 2.13.

```
X1      DB      100  DUP (2 DUP(?))   ; Описание входных
X2      DB      100  DUP(?)           ; массивов
Y       DB      100  DUP (2 DUP(?))   ; Описание массива частных
R       DB      100  DUP(?)           ; Описание массива остатков
        LEA     SI, X1 + type X1      ; Загрузка начальных адресов
        LEA     DI, X2
        LEA     DX, Y + type Y
        LEA     BP, R
        MOV     CX, length X1         ; Загрузка счетчика циклов
```

```
M1:     MOV     AL, [SI]            ; Чтение старшей цифры делимого
        MOV     AH, 0               ; Расширение старшей цифры
        AAD                         ; Коррекция делимого
        DIV     byte ptr [DI]       ; Деление на число X2
        MOV     [BX], AL            ; Запись старшей цифры
                                    ; результата
        MOV     AL, [SI-1]          ; Чтение младшей цифры
                                    ; делимого
        AAD                         ; Коррекция делимого
        DIV     byte ptr [DI]       ; Деление на число X2
        MOV     [BX – 1], AL        ; Запись младшей цифры
                                    ; результата
        MOV     DS: [BP], AH        ; Запись остатка
        DEC     CX                  ; Зацикливание
        JNZ     M1
```

2.3. Контрольные задачи

2.3.1. Вычислить Y = X + 1000, где X – число повышенной точности без знака.

2.3.2. Вычислить Y = X1 – X2, где X1, X2 – числа повышенной точности без знака.

2.3.3. Вычислить сумму элементов массива двойных слов без знака.

2.3.4. Вычислить сумму элементов массива двойных слов со знаком.

2.3.5. Вычислить Y = X1 + X2, где

а) X1, X2 – массивы слов без знака;

б) X1, X2 – массивы двойных слов без знака;

в) X1, X2 – массивы слов со знаком;

г) X1 – массив слов со знаком;
X2 – массив байт со знаком;

д) X1 – массив байтов без знака;
X2 – массив байтов со знаком.

2.3.6. Вычислить Y = X1 · X2, где

а) X1, X2 – массивы байтов без знака;

б) X1 – массив слов без знака;
X2 – массив байтов без знака;

в) X1 – массив байтов без знака;
X2 – массив байтов со знаком;

г) X1 – массив двойных слов без знака;

X2 – массив слов без знака;

д) X1, X2 – массивы двойных слов без знака;

е) X1, X2 – числа повышенной точности без знака.

2.3.7. Вычислить Y = X1/X2, где

а) X1, X2 – массивы слов без знака;

б) X1 – массив слов со знаком;

X2 – массив байтов без знака;

в) X1 – массив байтов без знака;

X2 – массив байтов со знаком;

г) X1 – массив квадрослов без знака;

X2 – массив слов без знака;

д) X1 – массив чисел повышенной точности без знака;

X2 – массив байтов без знака.

2.3.8. Вычислить Y = X1 + 525, где X1 – десятичное число повышенной точности в упакованном формате.

2.3.9. Вычислить Y = X1 – X2, где X1, X2 – десятичные числа повышенной точности в распакованном формате.

2.3.10. Найти сумму элементов массива 4-разрядных десятичных чисел в распакованном формате.

2.3.11. Вычислить Y = X1 ∗ X2, где X1 – десятичное число повышенной точности в распакованном формате, X2 – одноразрядное десятичное число в распакованном формате.

2.3.12. Вычислить Y = X1/X2, где X1 – 3-разрядное десятичное число в распакованном формате, X2 – одноразрядное десятичное число в распакованном формате.

3. ЛОГИЧЕСКИЕ КОМАНДЫ

В отличие от арифметических логические операции являются поразрядными, то есть в операцию вступают соответствующие биты операндов и порождают соответствующий бит результата.

3.1. Команды логических операций

Команда AND используется для выборочного обнуления полей операнда (выделения полей операнда), команда OR – выборочной установки полей операнда в 1 (объединение полей операндов), а команда XOR – для выборочного инвертирования полей операнда. При этом обрабатываемые поля и тип их обработки задаются положением и значением битов маски.

Задача 3.1. В памяти имеется массив слов без знака. В каждом элементе этого массива младшую тетраду обнулить, вторую установить в 1, третью проинвертировать, а старшую не изменять.

Решение 3.1.

```
ArrS    DW    100   DUP(?)        ; Описание входного массива
ArrD    DW    100   DUP(?)        ; Описание результата
        LEA   SI, ArrS            ; Загрузка начальных
        LEA   DI, ArrD            ; адресов
        MOV   CX, length ArrS     ; и счетчика циклов
M1:     MOV   AX, [SI]            ; чтение слова из памяти
        AND   AX, 0FFF0h          ; Обнуление младшей тетрады
        OR    AX, 0F0h            ; Установка 2-ой тетрады в 1
        XOR   AX, 0F00h           ; Инвертирование 3-ей тетрады
        MOV   [DI], AX            ; Запись результата
        ADD   SI, type ArrS       ; Модификация адресов
        ADD   DI, type ArrD
        DEC   CX                  ; Зацикливание
        JNZ   M1
```

Задача 3.2. В памяти имеется массив десятичных байтов в упакованном формате. Преобразовать все элементы этого массива в распакованный формат.

Решение 3.2.

Расположение данных в элементах исходного массива и результат их преобразования приведены на рис. 3.1.

a_1	a_0

0	a_0
0	a_1

Рис. 3.1. Размещение данных до и после распаковки

```
ArrS    DB      100     DUP(?)          ; Описание исходных данных
ArrD    DB      2*100   DUP(?)          ; Описание результата
        LEA     SI, ArrS                ; Загрузка адресов
        LEA     DI, ArrD
        MOV     CX, length ArrS         ; и счетчиков циклов
M1:     MOV     AL, [SI]                ; Чтение упакованного байта
        AND     AL, 0Fh                 ; Выделение младшей цифры a0
        MOV     [DI], AL                ; Запись младшей цифры
        MOV     AL, [SI]                ; Восстановление упакованного
        SHR     AL, 1                   ; байта
        SHR     AL, 1                   ; Сдвиг в младшую тетраду
        SHR     AL, 1                   ; с выделением старшей
        SHR     AL, 1                   ; цифры
        MOV     [DI + 1], AL            ; Запись старшей цифры
        INC     SI                      ; Модификация адресов
        ADD     DI, 2*type ArrD
        DEC     CX                      ; Зацикливание
        JNZ     M1
```

3.2. Команды сравнения

Задача 3.3. В памяти имеется массив слов со знаком. Выбрать из него все элементы большие (–3000).

Решение 3.3.

При формировании выходного массива путем выборки по некоторому критерию элементов из входного массива неизвестно сколько элементов удовлетворит заданному критерию. Поэтому память под выходной массив резервируется под максимальное значение элементов, но в начало выходного массива записывается значение количества выбранных элементов. Под количество элементов достаточно одного слова, поэтому выходной массив всегда на одно слово длиннее входного массива.

```
ArrS    DW      100          DUP (?)   ; Описание входного массива
ArrD    DW      (100 + 1)    DUP (?)   ; Описание выходного массива
        LEA     SI, ArrS               ; Загрузка начальных адресов
        LEA     DI ArrD
        ADD     DI, type ArrD          ; Смещение адреса за счетчик
                                       ; элементов
        MOV     CX, length ArrS        ; Загрузка счетчика циклов
        MOV     DX, 0                  ; Обнуление счетчика элементов
M2      MOV     AX, [SI]               ; Чтение элемента
        CMP     AX, –3000              ; Сравнение с заданным значением
        JLE     M1                     ; Переход, если меньше или равно
        MOV     [DI], AX               ; Выборка элемента
        INC     DX                     ; Подсчет выбранного элемента
        ADD     DI, type ArrD          ; Модификация адреса
                                       ; выходного массива
M1:     ADD     SI, type ArrS          ; Модификация адреса
                                       ; входного массива
        DEC     CX                     ; Зацикливание
        JNZ     M2
        MOV     ArrD, DX               ; Запись количества выбранных
                                       ; элементов
```

Задача 3.4. В памяти имеется массив слов со знаком. Выбрать из него все положительные значения, зафиксировать их местоположение.

Решение 3.4.

```
ArrX    DW      100          DUP (?)   ; Описание входного массива
ArrY    DW      (100 + 1)    DUP (?)   ; Описание массива
                                       ; выбираемых элементов
ArrA    DW      (100 + 1)    DUP (?)   ; Описание массива адресов
                                       ; выбираемых элементов
        LEA     SI, ArrX               ; Загрузка начальных адресов
        LEA     DI, ArrY
        LEA     BX, ArrA
        MOV     CX, length ArrX        ; Загрузка счетчика циклов
        MOV     DX, 0                  ; Обнуление счетчика элементов
M2:     MOV     AX, [SI]               ; Чтение элемента
        TEST    AX, 8000h              ; Проверка знака элемента
```

```
        JNZ     M1                  ; Переход, если отрицательный
        MOV     [DI + 2], AX        ; Запись элемента
        MOV     [BX + 2], SI        ; Запись адреса элемента
        INC     DX                  ; Подсчет элемента
        ADD     DI, type ArrY       ; Модификация адресов
        ADD     BX, type ArrA       ; выходных массивов
M1:     ADD     SI, type ArrX       ; Модификация адреса
                                    ; входного массива
        DEC     CX                  ; Зацикливание
        JNZ     M2
        MOV     ArrY, DX            ; Запись количества элементов
        MOV     ArrA, DX            ; в выходные массивы.
```

3.3. Команды сдвигов

Задача 3.5. В памяти имеется массив байтов со знаком. Выполнить быстрое умножение всех элементов этого массива на число 165.

Решение 3.5.

Представим число 165 в двоичной форме с последующим разложением по схеме Горнера.

$$165 = 10100101b = ((((((1\cdot 2 + 0)\cdot 2 + 1)\cdot 2 + 0)\cdot 2 + 0)\cdot 2 + 1)\cdot 2 + 0)\cdot 2 + 1$$

```
ArrS    DB      100     DUP(?)      ; Описание входного массива
ArrD    DW      100     DUP(?)      ; Описание выходного массива
        LEA     SI, ArrS            ; Загрузка начальных
        LEA     DI, ArrD            ; адресов
        MOV     CX, length ArrS     ; и счетчика циклов
M1:     MOV     AL, [SI]            ; Чтение элемента
        CBW                         ; Расширение элемента
        MOV     DX, AX              ; Копирование для последующих
                                    ; сложений
        SAL     AX, 1               ; Умножение на 2
        SAL     AX, 1               ; Умножение на 4
        ADD     AX, DX              ; Умножение на 5
        SAL     AX, 1               ; Умножение на 10
        SAL     AX, 1               ; Умножение на 20
        SAL     AX, 1               ; Умножение на 40
        ADD     AX, DX              ; Умножение на 41
```

```
        SAL     AX, 1               ; Умножение на 82
        SAL     AX, 1               ; Умножение на 164
        ADD     AX, DX              ; Умножение на 165
        MOV     [DI], AX            ; Запись результата
        ADD     SI, type ArrS       ; Модификация адресов
        ADD     DI, type ArrD
        LOOP    M1                  ; Зацикливание
```

Задача 3.6. В памяти имеется массив двойных слов без знака. Выполнить зеркальную перестройку битов во всех элементах этого массива.

Решение 3.6.

Для перестановки битов в числе используются команды циклического сдвига. Схема циклического сдвига двойных слов приведена на рис.3.2.

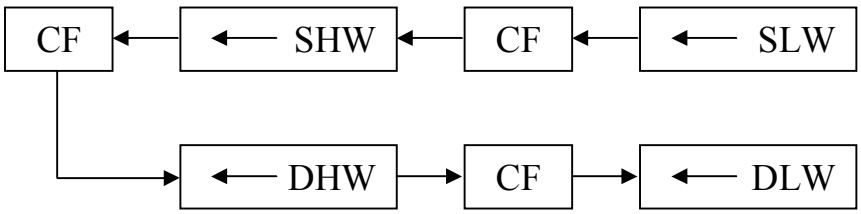


Рис.3.2. Циклический сдвиг двойных слов

Сначала выполняется сдвиг младшего (SLW) и старшего (SHW) слова входного элемента данных, а затем – старшего (DHW) и младшего (DLW) слова накопителя результата на 1 бит. На преобразование одного элемента массива требуется 32 сдвига.

```
ArrS    DD      100   DUP(?)        ; Описание входного
ArrD    DD      100   DUP(?)        ; и выходного массива
        LEA     SI, ArrS            ; Загрузка начальных
        LEA     DI, ArrD            ; адресов
        MOV     CX, length ArrS     ; и счетчика циклов
M2:     MOV     AX, [SI]            ; Чтение входного
        MOV     DX, [SI + 2]        ; элемента
        PUSH    CX                  ; Сохранение счетчика
                                    ; внешних циклов
        MOV     CX, 32              ; Загрузка счетчика
                                    ; внутренних циклов
M1:     RCL     AX, 1               ; Сдвиг входного элемента
```

```
RCL     DX, 1            ; на 1 бит
RCR     BX, 1            ; Сдвиг накопителя результата
RCR     BP, 1            ; на 1 бит
LOOP    M1               ; Зацикливание внутреннего цикла
MOV     [DI], BP         ; Запись элемента
MOV     [DI + 2], BX     ; результата
ADD     SI, type ArrS    ; Модификация адресов
ADD     DI, type ArrD
POP     CX               ; Восстановление счетчика
                         ; внешних циклов
LOOP    M2               ; Зацикливание внешнего цикла
```

3.4. Контрольные задачи

3.4.1. В памяти имеется массив десятичных чисел в распакованном формате. Преобразовать его в упакованный формат.

3.4.2. В памяти имеется массив байтов со знаком. Выполнить быстрое умножение всех элементов этого массива на число 227.

3.4.3. Найти сумму четных элементов массива слов без знака.

3.4.4. Из массива слов со знаком выбрать все элементы большие (–5000), но меньшие (+5000). Зафиксировать местонахождение выбираемых элементов.

3.4.5. В памяти имеется массив двухразрядных десятичных чисел в упакованном формате. Выполнить перестановку цифр во всех байтах этого массива, старшей цифрой в которых является 7.

3.4.6. Определить количество элементов со значением 3251 в массиве слов без знака.

3.4.7. Из массива слов без знака выбрать все элементы со значением поля D13 ÷ D9 = 10101.

3.4.8. Из массива 20-разрядных десятичных чисел в упакованном формате выбрать элементы кратные 10.

3.4.9. Найти сумму элементов массива 6-разрядных десятичных чисел в упакованном формате, старшая цифра которых равна 5.

3.4.10. Из массива 20-разрядных десятичных чисел в распакованном формате выбрать все элементы, кратные 100.

4. ОРГАНИЗАЦИЯ ПОДПРОГРАММ

4.1. Передача параметров в подпрограммы

Передача параметров в подпрограммы дает возможность с помощью одной и той же подпрограммы обрабатывать различные данные. Для этого на макроуровне перед вызовом подпрограммы необходимо передать в нее параметры, определяющие фактические данные, которые будут обрабатываться при этом вызове. В качестве фактических параметров чаще всего используются указатели на данные: адрес и длина массивов, а передача параметров осуществляется через стек.

Здесь и далее текст программы, решающий поставленную задачу, будет записываться по полной форме.

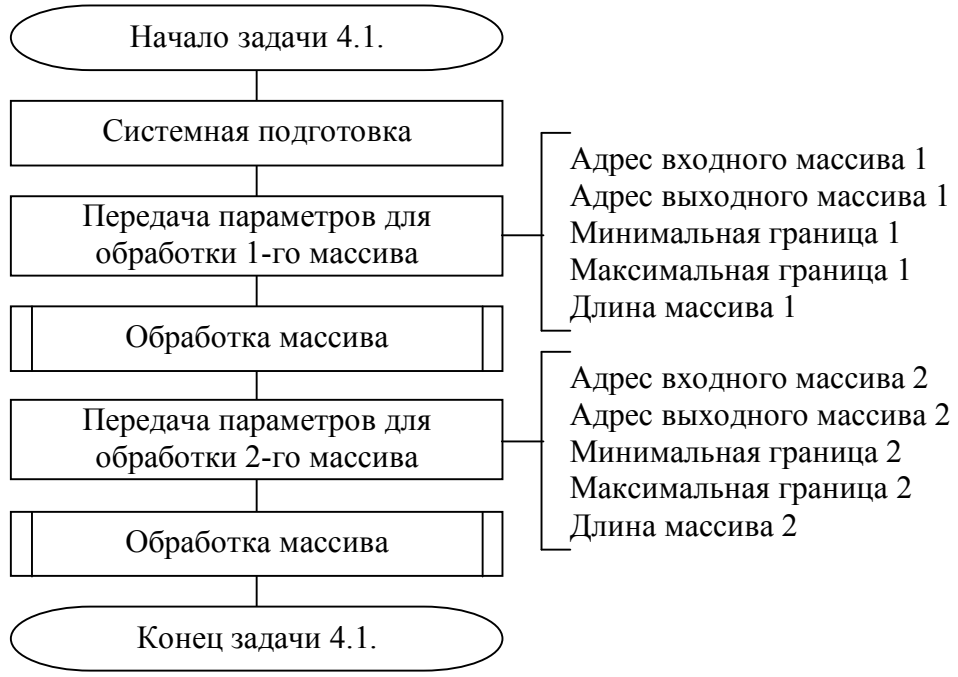
Задача 4.1. Написать подпрограмму для выборки из некоторого массива слов без знака всех элементов больших одного, но меньших другого значения. С помощью этой подпрограммы обработать два массива, выбирая из первого элементы большие (–5000), но меньшие 5000, а из второго – большие 10000, но меньшие 20000. Для передачи параметров использовать стек.

Решение 4.1.

Из условия задачи следует, что подпрограмма должна обрабатывать два массива. Следовательно, она должна работать с передачей параметров, используя заданный способ передачи.

Входные данные представляют собой массив слов без знака, из которого выбираются элементы, удовлетворяющие заданному условию. Следовательно, выходной массив также будет представлять собой массив слов без знака. Длина выходного массива будет на один элемент больше длины входного массива. Этот элемент предназначен для хранения числа, определяющего количество фактически выбранных элементов, и должен находиться в начале выходного массива.

В общем случае, при решении сложных задач сначала разрабатывается алгоритм обработки данных, а затем пишется исходный текст программы. Полный алгоритм включает в себе алгоритм макроуровня программы и алгоритм работы подпрограммы. Алгоритм решения задачи 4.1. приведен на рис. 4.1.



а)

б)

Рис. 4.1. Алгоритм решения задачи 4.1
а) алгоритм макроуровня б) алгоритм подпрограммы

Для полного написания программы необходимо знать смещения фактических параметров относительно верхушки стека. Смещения зависят от последовательности записи параметров в стек, выполняемой на макроуровне программы. Поэтому при написании программы после описания обрабатываемых данных сначала пишется макроуровень программы, и при этом составляется схема их размещения в стеке. Далее ячейки стека нумеруются от верхушки и пишется подпрограмма.

В данной задаче параметрами являются указатели на входные и выходные массивы данных, а также константы, определяющие границы заданного диапазона. Значения фактических параметров передаются в стек в последовательности, указанной на рис. 4.1 и исходном тексте макроуровня. Это позволяет составить схему размещения параметров в стеке, приведенную на рис. 4.2.

	BP	0
	IP	2
	Длина	4
	Max	6
	Min	8
	Адрес выходного массива	10
	Адрес входного массива	12
$(SP)_{исх}$		

Рис.4.2. Схема размещения параметров в стеке

В схеме на рис. 4.2 также учтено, что в начале подпрограммы в стеке сохраняется регистр BP, что смещает его верхушку на одно слово.

На основании алгоритма (см. рис. 4.1) и схемы размещения параметров в стеке (см. рис. 4.2) можно полностью написать исходный текст программы.

```
        Name VyborTask          ; Имя программы
        .286        ; Подключение дополнительных команд 80286
Min1    =           –5000       ; Описание констант
Max1    =           5000
Min2    =           10000
Max2    =           20000
Data    Segment                 ; Сегмент данных
```

```
        Arr1S   DW      100         DUP(?)  ; Описание входных
        Arr2S   DW      200         DUP(?)  ; массивов
        Arr1D   DW      (100 + 1)   DUP(?)  ; Описание выходных
        Arr2D   DW      (200 + 1)   DUP(?)  ; массивов
Data            EndS
Stack           Segment                     ; Сегмент стека
                DW      16          DUP(?)
        StkTop  Label   word                ; Фиксация верхушки
Stack           EndS                        ; пустого стека
Code            Segment                     ; Сегмент программного кода
                Assume CS: Code, DS:Data, SS: Stack

                ; Описание подпрограммы
        Vybor   Proc
                PUSH    BP
                MOV     BP, SP          ; Привязка к верхушке стека
                MOV     CX, [BP + 4]    ; Чтение параметров
                MOV     SI, [BP + 12]   ; из стека
                MOV     DI, [BP + 10]
                ADD     DI, 2           ; Смещение адреса выходного
                                        ; массива за счетчик элементов
                XOR     DX, DX          ; Обнуление счетчика элементов
        M2:     MOV     AX, [SI]        ; Чтение элемента
                CMP     AX, [BP + 8]    ; Сравнение с нижней границей
                JBE     M1              ; Переход, если меньше или равно
                CMP     AX, [BP + 6]    ; Сравнение с верхней границей
                JAE     M1              ; Переход, если больше или равно
                MOV     [DI], AX        ; Выборка элемента
                INC     DX              ; Подсчет выбранного элемента
                ADD     DI, 2           ; Модификация адреса выходного
                                        ; массива
        M1:     ADD     SI, 2           ; Модификация адреса входного
                                        ; массива
                LOOP    M2
                MOV     DI, [BP + 10]   ; Восстановление адреса
                                        ; выходного массива
                MOV     [DI], DX        ; Запись количества
                                        ; выбранных элементов
```

```
        RET     10              ; Возврат с исключением
Vybor   Endp                    ; параметров из стека

        ; Макроуровень программы
Start:  MOV     AX, Data        ; Системная подготовка:
        MOV     DS, AX          ; инициализация
        MOV     AX, Stack       ; сегментных
        MOV     SS, AX          ; регистров и указателя
        LEA     SP, StkTop      ; стека
        PUSH    offset Arr1S    ; Передача параметров
        PUSH    offset Arr1D    ; первого
        PUSH    Min1            ; массива в стек
        PUSH    Max1            ; (см. рис. 4.2)
        PUSH    length Arr1S
        CALL    Vybor           ; Обработка массива
        PUSH    offset Arr2S    ; Передача параметров
        PUSH    offset Arr2D    ; второго массива
        PUSH    Min2            ; в стек
        PUSH    Max2            ; (см. рис. 4.2)
        PUSH    length Arr2S
        CALL    Vybor           ; Обработка массива
Code    Ends
End     Start                   ; Конец исходного текста
                                ; программы
```

4.2. Контрольные задачи

4.2.1. Написать подпрограмму для поиска и фиксации местоположения заданного значения в некотором массиве слов без знака. С помощью этой подпрограммы обработать два массива, отыскивая в первом элементы со значением 5231, а во втором – элементы со значением 45783. Для передачи параметров использовать стек.

4.2.2. Написать подпрограмму для выборки из некоторого массива слов без знака всех элементов с заданным значением заданного поля. С помощью этой подпрограммы обработать два массива, выбирая из первого элементы со значением поля D13 ÷ D10 = 1011, а из второго – элементы со значением поля D7 ÷ D2 = 110110.

5. ОРГАНИЗАЦИЯ МАКРОКОМАНД

5.1. Передача параметров в макрокоманды

Макрокоманды также как и подпрограммы позволяют сократить длину исходного текста программы. Путем передачи параметров с помощью одной и той же макрокоманды можно обрабатывать различные данные. При написании программы сначала дается макроопределение макрокоманды относительно формальных параметров, а затем осуществляются макровызовы с передачей фактических параметров.

Задача 5.1. Написать макрокоманду для выборки из некоторого массива слов без знака всех элементов больших одного, но меньших другого значения. С помощью этой макрокоманды обработать два массива, выбирая из первого элементы большие (–5000), но меньшие 5000, а из второго – большие 10000, но меньшие 20000.

Решение 5.1.

По своей сущности эта задача совпадает с задачей 4.1. Поэтому совпадает и представление входных и выходных данных в этих задачах.

Полный алгоритм задачи также включает в себя алгоритм макроуровня и алгоритм макрокоманды, приведенные на рис. 5.1.

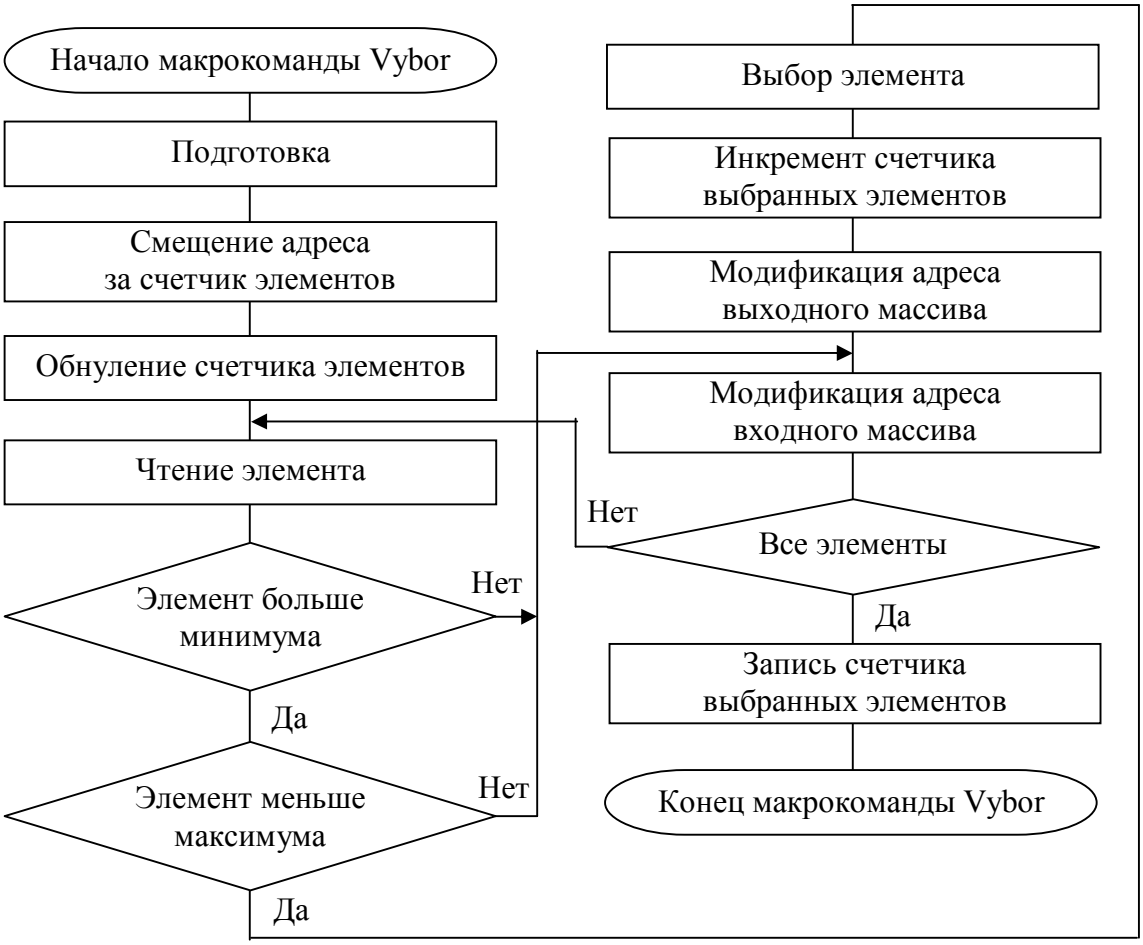
Параметры в макрокоманду передаются в текстовом виде. Это позволяет писать макроопределение относительно формальных параметров точно так, как относительно фактических параметров, что существенно упрощает передачу параметров. Передача фактических параметров осуществляется при макровызове путем их простого перечисления в макрокоманде.

```
                Name VyborTask
        Min1    =       –5000                   ; Описание констант
        Max1    =       5000
        Min2    =       10000
        Max2    =       20000
Data            Segment                         ; Сегмент данных
        Arr1S   DW      100         DUP(?)      ; Описание входных
        Arr2S   DW      200         DUP(?)      ;массивов
        Arr1D   DW      (100 + 1)   DUP(?)      ;Описание выходных
        Arr2D   DW      (200 + 1)   DUP(?)      ; массивов
Data            Ends
Code            Segment                         ; Сегмент программного кода
                Assume CS: Code, DS: Data
```

```
                ; Макроопределение
        Vybor   Macro ArrS, ArrD, Min, Max
                LOCAL   M1, M2              ; Локализация меток
                LEA     SI, ArrS            ; Загрузка начальных
                LEA     DI, ArrD            ; адресов
                ADD     DI, type ArrD       ; Смещение адреса за счетчик
                                            ; элементов
                MOV     CX, length ArrS     ; Загрузка счетчика циклов
                XOR     DX, DX              ; Обнуление счетчика элементов
        M2:     MOV     AX, [SI]            ; Чтение элемента
                CMP     AX, Min             ; Сравнение с нижней границей
                JBE     M1                  ; Переход, если меньше или равно
                CMP     AX, Max             ; Сравнение с верхней границей
                JAE     M1                  ; Переход, если больше или равно
                MOV     [DI], AX            ; Выборка элемента
                INC     DX                  ; Подсчет выбранного элемента
                ADD     DI, type ArrD       ; Модификация адреса
                                            ; выходного массива
        M1:     ADD     SI, type ArrS       ; Модификация адреса
                LOOP    M2                  ; входного массива
                MOV     ArrD, DX            ; Запись количества
                                            ; выбранных элементов
                Endm

                ; Макроуровень программы
        Start:  MOV     AX, Data            ; Системная подготовка
                MOV     DS, AX
                ; Обработка первого массива
                Vybor Arr1S, Arr1D, Min1, Max1   ; Макровызов
                ; Обработка второго массива
                Vybor Arr2S, Arr2D, Min2, Max2   ; Макровызов
Code            Ends
End             Start
```

а)



б)

Рис. 5.1. Алгоритм решения задачи 5.1
а) алгоритм макроуровня б) алгоритм макрокоманды

5.2. Контрольные задачи

5.2.1. Написать макрокоманду для поиска и фиксации местоположения заданного значения в некотором массиве слов без знака. С помощью этой макрокоманды обработать два массива, отыскивая в первом элементы со значением 5231, а во втором – элементы со значением 45783.

5.2.2. Написать макрокоманду для выборки из некоторого массива слов без знака всех элементов с заданным значением заданного поля. С помощью этой макрокоманды обработать два массива, выбирая из первого элементы со значением поля D13 ÷ D10 = 1011, а из второго – элементы со значением поля D7 ÷ D2 = 110110.

6. ОБРАБОТКА ДВУМЕРНЫХ МАССИВОВ

6.1. Размещение в памяти и обработка

Каждый элемент двухмерного массива A(I,J) характеризуется двумя номерами: номером строки I и номером столбца J. Однако, память представляет собой одномерный массив байтов. В связи с этим двухмерный массив может храниться в ней по строкам или по столбцам. Различные варианты хранения двухмерного массива A

A(0,0)	A(0,1)	. . . .	A(0,M)
A(1,0)	A(1,1)	. . . .	A(1,M)
. . . . .			
A(L,0)	A(L,1)	. . . .	A(L,M)

размерностью (L*M) приведены на рис. 6.1.

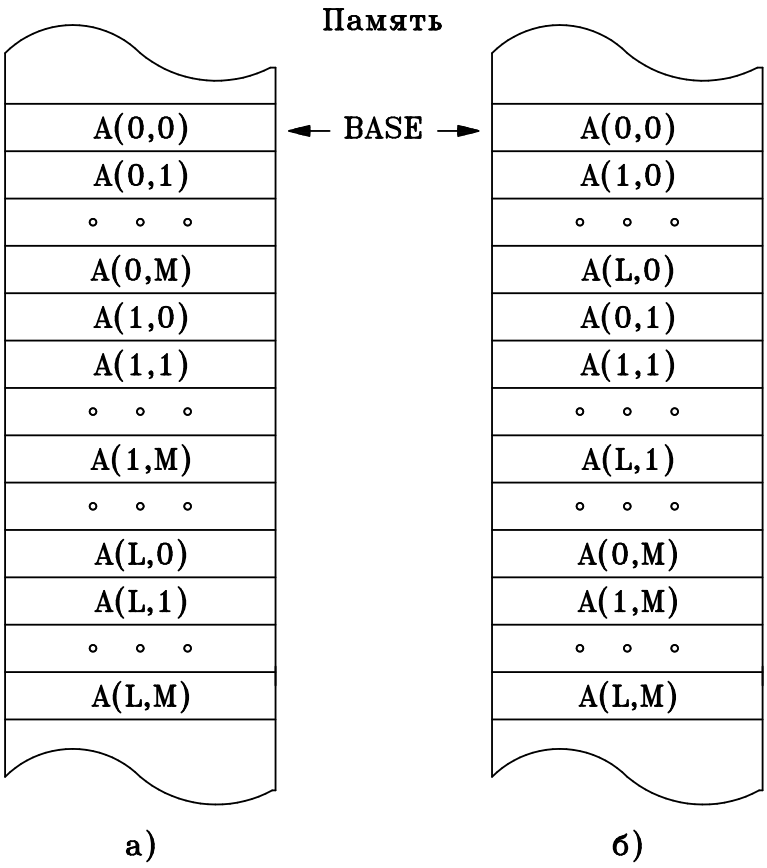


Рис. 6.1. Размещение двухмерного массива в памяти:
а) по строкам; б) по столбцам

Для определения адреса некоторого элемента массива A(I, J) необходимо вычислить его общий индекс по формулам:

$$IND(I, J) = I * M + J \quad \text{– при хранении по строкам;}$$
$$IND(I, J) = I + J * L \quad \text{– при хранении по столбцам.}$$

После этого адрес элемента A(I, J) двухмерного массива определяется обычным образом по формуле

$$ADDR(I, J) = BASE + IND(I, J) * type\ A,$$

где BASE – начальный адрес массива.

Обработка двумерного массива также выполняется циклически. При этом двумерный массив представляет собой одномерный массив одномерных массивов. Следовательно, оператор “Тело цикла” в схеме циклической обработки одномерного массива (см. рис. 1.3) должен быть реализован по той же самой циклической схеме. Подставляя полную схему циклической обработки одномерного массива в ее оператор “Тело цикла”, можно получить алгоритм циклической обработки двумерного массива, приведенный на рис. 6.2.

Рис. 6.2. Алгоритм циклической обработки двумерного массива

Для адресации элементов двумерного массива, как правило, используют базово-индексную адресацию. При этом в базовые регистры помещают начальные адреса строк, а в индексные регистры – адреса столбцов.

Задача 6.1. В памяти имеется двумерный массив слов. Скопировать его элементы, находящиеся на пересечении строк и столбцов с нечетными индексами в выходной массив.

```
        N1=100
        N2=200
ArrX    DW      N1    DUP(N2 DUP(?))          ; Входной массив
ArrY    DW      N1/2 DUP(N2/2 DUP(?))         ; Выходной массив
        MOV     CX, N1/2                      ; Загрузка счетчика
                                              ; внешнего цикла
        LEA     BX, ArrX + N2*type Arr X      ; Загрузка базы входного
                                              ; массива на строку
                                              ; с нечетным индексом
        LEA     BP, ArrY                      ; Загрузка базы
                                              ; выходного массива
M2:     PUSH    CX                            ; Сохранение счетчика
                                              ; внешних циклов
        MOV     CX, N2/2                      ; Загрузка счетчика
                                              ; внутренних циклов
        MOV     SI, type ArrX                 ; Загрузка адреса столбца
                                              ; входного массива
                                              ; на нечетный индекс
        MOV     DI, 0                         ; Загрузка индекса
                                              ; выходного массива
M1:     MOV     AX, [BX + SI]                 ; Чтение элемента
                                              ; входного массива
        MOV     DS: [BP + DI], AX             ; Запись элемента
                                              ; выходного массива
        ADD     SI, 2*type ArrX               ; Модификация адреса столбца
                                              ; входного массива
                                              ; через элемент
        ADD     DI, type ArrY                 ; Модификация адреса
                                              ; столбца выходного массива
        LOOP    M1                            ; Зацикливание внутреннего
                                              ; цикла
```

```
        ADD     BX, 2*N2*type ArrX          ; Модификация адреса строки
                                            ; входного массива
        ADD     BP, (N2/2)*type ArrY        ; Модификация адреса
                                            ; строки выходного массива
        POP     CX                          ; Восстановление счетчика
                                            ; внешних циклов
        LOOP    M2                          ; Зацикливание внешнего цикла
```

К обработке двумерных массивов сводится выполнение многих операций над числами повышенной точности.

Задача 6.2. Вычислить Y = X1/X2, где X1 – массив чисел повышенной точности без знака, X2 – массив слов без знака.

Решение 6.2. Массив X1 – это двумерный массив, т.к. его элементами являются числа повышенной точности, каждое из которых представляется одномерным массивом. В соответствии с этим и массив Y будет двумерным массивом, т.к. частное от деления каждого элемента также будет числом повышенной точности. В соответствии с правилами арифметики деления каждого числа повышенной точности из массива X1 должно начинаться со старшей цифры. При этом и частное будет формироваться со старшей цифры. Следовательно, для адресации элемента делимого и частного может использоваться один и тот же индексный регистр. Алгоритм решения этой задачи приведен на рис. 6.3.

```
        N1=100
        N2=200
X1      DW      N1  DUP(N2  DUP(?))     ; Описание делимого
X2      DW      N1  DUP(?)              ; Описание делителя
Y       DW      N1  DUP(N2  DUP(?))     ; Описание частного
R       DW      N1  DUP(?)              ; Описание остатка
        LEA     BX, X1                  ; Загрузка базовых адресов X1
        LEA     BP, Y                   ; и Y
        LEA     DI, X2                  ; Загрузка начальных адресов X2
        LEA     SI, R                   ; и остатков
        MOV     CX, N1                  ; Загрузка счетчика внешних циклов
M2:     PUSH    CX                      ; Загрузка счетчика
        MOV     CX, N2                  ; внутренних циклов
        PUSH    SI                      ; Загрузка индекса делимого
        MOV     SI, (N2 – 1)*type X1    ; и частного на старший элемент
        MOV     DX, 0                   ; Обнуление старшей части
                                        ; первого элемента делимого
```

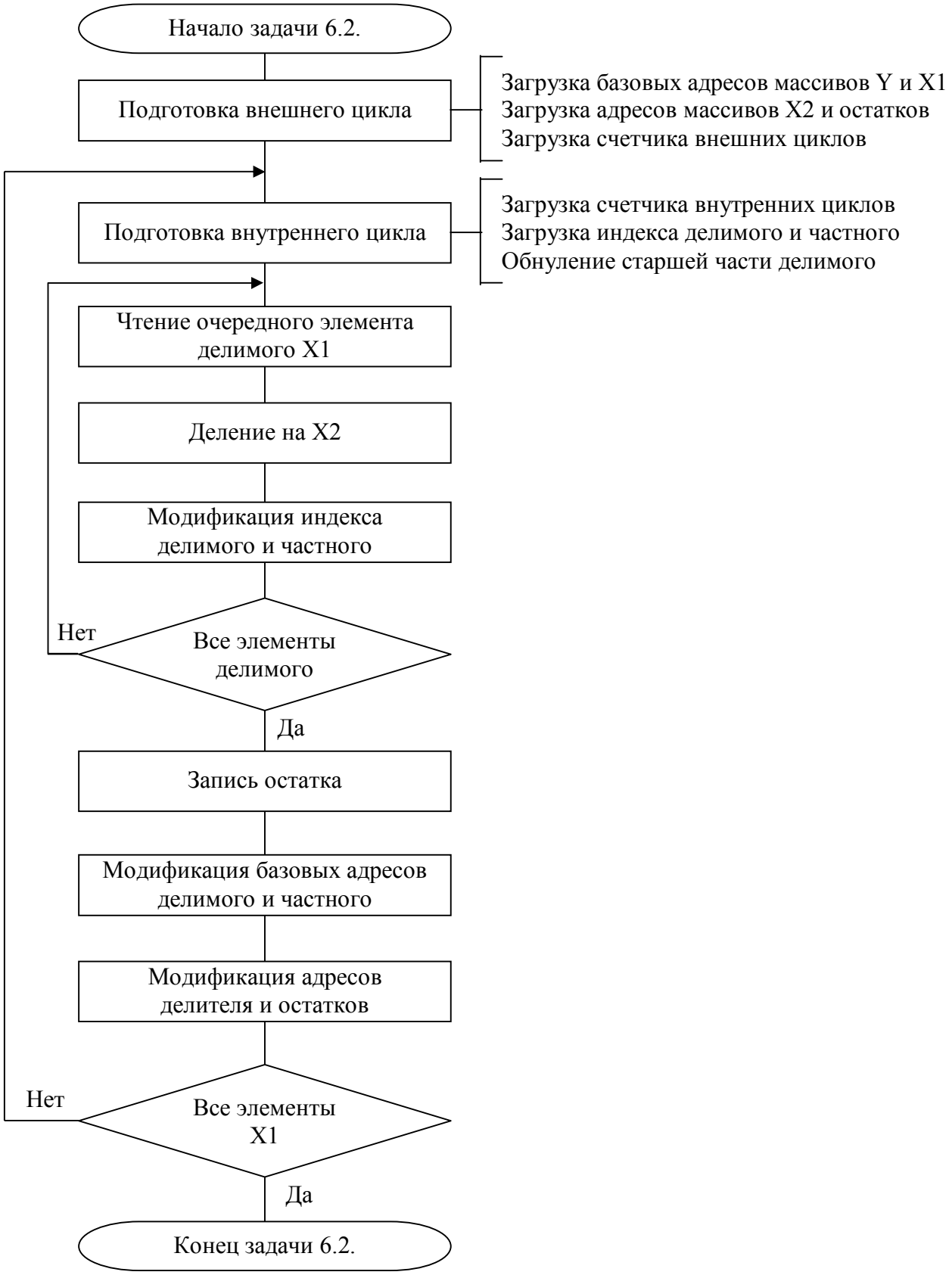


Рис. 6.3. Алгоритм решения задачи 6.2

```
M1:  MOV    AX, [BX + SI]          ; Чтение элемента делимого
     DIV    word ptr [DI]          ; Деление на X2
     MOV    DS:[BP + SI], AX       ; Запись элемента частного
     SUB    SI, type X1            ; Модификация индекса
                                   ; делимого и частного
     LOOP   M1                     ; Зацикливание внутреннего цикла
     POP    SI                     ; Запись остатка
     MOV    [SI], DX
     ADD    BX, N2*typeX1          ; Модификация базовых адресов
     ADD    BP, N2*type Y          ; X1 и Y
     ADD    DI, type X2            ; Модификация
     ADD    SI, type R             ; адресов X2 и R
     POP    CX                     ; Зацикливание
     LOOP   M2                     ; внешнего цикла
```

6.2. Контрольные задачи

6.2.1. В памяти имеется двумерный массив байтов. В каждую третью строку этого массива скопировать строку символов “Ассемблер – это язык программирования, обеспечивающий высокую эффективность создаваемых программ по памяти и времени выполнения”, инициализированную в памяти системы.

6.2.2. В памяти имеется массив десятичных чисел повышенной точности в упакованном формате. Выбрать из этого массива все числа, кратные 1000.

ЗАКЛЮЧЕНИЕ

В настоящем пособии рассмотрено решение большинства типовых задач программирования на ассемблере. Анализируя приведенные решения необходимо выявить в них общие закономерности, чтобы использовать их при решении подобных задач. Для закрепления материала необходимо самостоятельно решить задачи по всем рассмотренным темам. Задачи для самостоятельного решения приведены в конце каждого раздела.

СПИСОК РЕКОМЕНДУЕМЫХ ИСТОЧНИКОВ

Комаров В.М. Микропроцессорные системы. Программирование на ассемблере: Учебное пособие/РГАТА – Рыбинск, 2004 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. ОБЩАЯ ОРГАНИЗАЦИЯ ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ. КОМАНДЫ ПЕРЕСЫЛКИ ДАННЫХ 4
1.1. Описание данных 4
1.2. Команды пересылки данных 6
1.2.1. Команды пересылки общего назначения 6
1.2.2. Команды загрузки адресных значений и обращения к стеку 11
1.2.3. Команды ввода/вывода 13
1.3. Контрольные задачи 14
2. АРИФМЕТИЧЕСКИЕ КОМАНДЫ 15
2.1. Арифметические операции над двоичными числами 15
2.1.1. Сложение (вычитание) двоичных чисел без знака 15
2.1.2. Сложение (вычитание) двоичных чисел со знаком 17
2.1.3. Умножение двоичных чисел 18
2.1.4. Деление двоичных чисел 22
2.2. Арифметические операции над десятичными числами. 24
2.2.1. Сложение десятичных чисел 25
2.2.2. Умножение десятичных чисел 27
2.2.3. Деление десятичных чисел 27
2.3. Контрольные задачи 28
3. ЛОГИЧЕСКИЕ КОМАНДЫ 30
3.1. Команды логических операций 30
3.2. Команды сравнения 31
3.3. Команды сдвигов 33
3.4. Контрольные задачи 35
4. ОРГАНИЗАЦИЯ ПОДПРОГРАММ 36
4.1. Передача параметров в подпрограммы 36
4.2. Контрольные задачи 40
5. ОРГАНИЗАЦИЯ МАКРОКОМАНД 41
5.1. Передача параметров в макрокоманды 41
5.2. Контрольные задачи 44
6. ОБРАБОТКА ДВУМЕРНЫХ МАССИВОВ 45
6.1. Размещение в памяти и обработка 45
6.2. Контрольные задачи 50
ЗАКЛЮЧЕНИЕ 51
СПИСОК РЕКОМЕНДУЕМЫХ ИСТОЧНИКОВ 51